

SLAM 3.0 API

Serial communication

```
1  Serial port baud rate: 115200
2  Navigation host serial port: /dev/XX
3  **Command package:**
4  - The communication method is a serial asynchronous method, and the communication
    content between the ROS system and the host computer is an ASCII coded string.
5
6  | Frame Header (H) | Length (L) | Data (D) | Check Digit (S)|
7  |:---| :-----:| :-----:| :-----:|
8  |AA 54| |||||
9
10
11 **Instruction package data definition:**
12 - Frame header (H): AA 54
13 - Length (L): length of data (D)
14 - Data (D): byte array converted from string
15 - Parity bit (S): XOR calculation of length (L) and data (D)
16
17
18 **Response Packet:**
19 - The communication method is a serial asynchronous method, and the communication
    content between the ROS system and the host computer is an ASCII coded string.
20
21 | Frame Header (H) | Length (L) | Data (D) | Check Digit (S)|
22 |: | :-----:| :-----:| :-----:|
23 |AA 54| |||||
24
25
26 **Response packet data definition:**
27 - Frame header (H): AA 54
28 - Length (L): length of data (D)
29 - Data (D): byte array converted from string
30 - Parity bit (S): XOR calculation of length (L) and data (D)
```

ROS actively reports

1. imu, laser, odometer, 3d camera status

```
1 response: check_sensors{imu lidar odom 3dcam ros}
2 mark:
3 imu: imu state;
4 lidar: Lidar status;
5 odom: odometer status;
6 3dcam: 3d camera status;
7 ros: ros service status;
8 0: Abnormal; 1: Normal;
9 The navigation system actively reports, once every 5s
```

2. navigation state

```
1 response:
2 nav_result{state code name dist_to_goal mileage}
3 mark:
4 state:
5     0: initial state;
6     1: The machine starts to navigate;
7     2: Pause;
8     3: Navigation is complete;
9     4: Cancellation of navigation;
10    5: Navigation recovery;
11    6: The upper computer sends the navigation command successfully, but the
    navigation has not started yet;
12    Navigation process:
13        send : nav_point[A]//send navigation target point
14        resp : nav_result{6 0 A -1 0}//Start navigation
15        resp : nav_result{1 0 A 0.562001 0}
16        resp : nav_result{3 0 A 0 0}//navigation completed
17        resp : nav_result{0 0 A -1 0}
18 code:
19     When sending navigation commands:
20     0 : success;
21     -1 : docking charging pile;
22     -2 : emergency stop switch is pressed;
23     -3 : The adapter is charging;
24     -4 : target point not found;
25     -5: AGV docking failed;
26     -6: Abnormal positioning;
27     -7: The distance between fixed route points is too large;
28     -8: No fixed route found;
29     -9: Failed to read point information;
30     Send pause/resume navigation command:
```

```
31      0 : success;
32      -1 : failed;
33      Send Cancel Navigation: 0;
34      Navigation complete:
35          0 : success;
36          -1 : failed;
37      name:
38          Target point navigation: target point name;
39          Coordinate Navigation :
40              x: x-axis coordinates;
41              y : y-axis coordinate;
42              radian : radian
43      dist_to_goal : the distance from the target point;
44      mileage : navigation mileage;
```

3. Navigation error state

```
1 response: move_status:x
2 mark:
3 x:
4: Unable to plan the route;
4: There are obstacles in the local path;
5: Automatically restart navigation if single navigation fails (fixed route)
6: Encounter an obstacle and prepare to start walking along the obstacle course
```

4. core state

```
1 response: core_data{data1 data2 data3 data4 data5}
2 mark:
3   data1: Collision:
4     1: center; 2: back; 4: left; 8: right;
5   data2: anti-drop:
6     4: left; 2: center; 1: 3D;
7   data3: emergency stop:
8     0: press the release axis; 1: lift up the navigation;
9   data4: Power:
10    battery percentage;
11   data5: charging status:
12    1: Not charging; 2: Charger; 3: Adapter; 8: Docking; 9: The charging pile was not
    found when docking; 11: The infrared code was not received after timeout; 12: The infrared
    code was not received in the second stage of the docking charging pile ; 13: The third
    stage of docking charging pile did not receive the infrared code
```

5. wheel status

```

1  response:
2      wheel_status{work_state current_left current_right temp_left temp_right
driver_temp_left driver_temp_right code_left code_right model version}
3  mark:
4      work_state : work state
5          loose axis 0x00
6          Shaft lock 0x01
7          Wheel failure 0x02
8          brake 0x04
9          stop (stop) 0x08
10         Offline status 0x80
11     current_left : left wheel current (mA);
12     current_right : right wheel current (mA);
13     temp_left : left wheel temperature (°C);
14     temp_right : right wheel temperature (°C);
15     driver_temp_left : left wheel driver temperature (°C);
16     driver_temp_right : right wheel drive temperature (°C);
17     code_left : left wheel error code;
18     code_right : right wheel error code;
19     model : wheel model;
20
21 version : wheel version number;

```

6. point update

```

1  response: waypoint:update
2  mark: report when the point is updated

```

7. Fixed route updates

```

1  response: pathmodel:update
2  mark: report when the fixed route is updated (not supported before 3.2.2)

```

8. shutdown

```

1  response: power_off:x
2  mark:
3      x: 1 : software shutdown; 2 : restart; 3 : button shutdown; 4 : low power shutdown;
4  Android shutdown is required upon receipt of a shutdown report;

```

9. Report speed during navigation

```

1  response: base_vel[line_speed angular_speed]
2  mark:
3      line_speed : line speed;
4      angular_speed : angular speed;

```

10. Report the rear ultrasonic wave when docking with the charging pile

```
1 response: range_sensor{data1 data2 data3}
2 mark:
3     data1 : post-ultrasonic;
```

11. global path

```
1 response: global_path: & global_path1:
2 mark:
3     global_path: global path, if it cannot be fully reported once, it will be reported twice
    (global_path will end with +)
```

12. agv coordinate reporting

```
1 response: agv_tag_pose{tag_name pose1 pose2 pose3 poseX,poseY,poseAngle}
2 mark:
3     tag_name:tag code name;
4     poseX:X-axis offset;
5     poseY:Y-axis offset;
6     poseAngle:angle;
```

13. agv navigation results

When performing agv navigation, ros will first report nav_result, indicating that navigation to the target point is successful, then adjust the position according to the QR code, and then perform QR code navigation. Here is the result of QR code navigation;

```
1 agv navigation success:
2     response: agv_success{tag}
3     mark:
4         agv navigation success;
5         tag: target tag code;
6 agv navigation failure:
7     response:agv_fail
8     mark:
9         agv navigation failure;
```

14. agv navigation results

```
1 response: misspose:code
2 mark:
3     code:
4         0: reposition, position reset
5         1: position lost
```

15. Generate fixed route

```
1 request: short_dij:& short_dij1:
2 mark:
3     If the report cannot be completed in one time, it will be reported in multiple times (if
    the report is not completed, it will end with +)
```

16. Report special zones that global paths pass through

```
1 response: special_plan:{"sp":[{"n":"sp-b","c":[-6.03,-0,-8.66,-2.08],"type":0}]}
2 mark:
3     n: special zone name;
4     c: intersection coordinates of global route and special zone;
5     type: special zone type:
6         0: default;
7         1: one-way zone;
8         2: gated zone;
9         3: turning zone;
10        4: custom;

1 response: special_area[sp-a,0,-1.0]
2 mark:
3     sp-a: special area name;
4     0: special area type;
5     -1.0: speed when passing through the special area, -1 is the default speed;
```

17. Leaving the Special Zone

```
1 response: special_area:out
```

Unique to Snail Sweeper

1. Cleaning progress

```
1 response: clean_room[running:progress,cleanroom_id]
2 mark:
3     progress: progress;
```

2. Cleaning completed

```
1 response: clean_room[complete:code,clean_id]
2 mark:
3     code:
4         0:completed;
```

Unique to Forklift

1. Forklift related status

```
1 response: forklift{arm_location position_sensor dock_state control_state reserved}
2     arm_location: forklift arm position
3         0: middle;
4         1: top;
5         2: bottom;
6         3: abnormal;
7     position_sensor: pallet in place sensor
8         0: not in place;
9         1: in place;
10    dock_state: docking state
11        0: initial state;
12        1: docking;
13        2: docking successful;
14        3: lateral deviation adjustment successful;
15        4: visual recognition of pallet not available;
16        5: near-end verification failed;
17        6: collision with pallet;
18        7: obstacle avoidance failed;
19    control_state: forklift control state
20        0: ros control;
21        1: manual control;
22    reserved: reserved position;
```

The host computer actively requests

1. heartbeat packet

```
1 request: keep_connect
2 response:
3     hfls_version: HardwareVersion FirmwareVersion LoaderVersion SoftVersion
4 mark:
5     HardwareVersion power board version number, FirmwareVersion power board
6     firmware version number, LoaderVersion: loaderVersion, SoftVersion navigation system
    version number
6 Keep requesting once every 5 seconds, and the watchdog of the navigation system will
    check the connection status of the serial port
```

2. Shut down the whole machine

```
1 request: power_off
2 response: power_off: 1
```

3. restart navigation

```
1 request: sys:reboot
2 response: initpose: 0, xy radian
3 mark:
4     The navigation system will actively report when starting up/restarting the
    navigation/switching the map (in these cases, the navigation will be relocated;
5 xy radian: machine coordinates;
```

4. Laser data reporting

```
1 request: switch_lidar[open?on:off]
2 response: laser[distance]
3 mark:
4   open: on : open; off : close;
5   distance : distance, unit: m, accurate to cm;
```

5. Navigation host connected to wifi

```
1 request: wifi[ssid name;pwd password]
2 response:
3   wifi:connect success The connection is successful
4   wifi:connect fail connection failed
5   wifi:connecting connection in progress
6 mark:
7   name is the WIFI name
8   password is the WIFI password
9   WIFI only supports WPA/WPA2 Personal protocol
```

6. Get the navigation host Hostname

```
1 request: hostname:get
2 response: sys:boot:robot-18
3 mark:
4   robot-18 is the Hostname of the navigation host
```

7. Get navigation system version

```
1 request: sys:version
2 response: ver:3.0.0
```

8. Get navigation wifi name and ip

```

1 request: ip:request
2 response: ip:ssid:xxxx/wlan:127.0.0.1
3 mark:
4   ssid:wifi name
5   xxxx: ip
6   wlan:127.0.0.1: network not connected

```

9. Query network port ip

```

1 request: ip_lan: request
2 response: ip:enp2s0:192.168.10.2 / ip:lan:127.0.0.1

```

10. get current map

```

1 request: nav:current_map
2 response: current_map[map alias]
3 mark:
4   map: current map md5
5   alias: alias (if there is no alias response:current_map[map])

```

11. query navigation mode

```

1 request: model: request
2 response: model:x
3 mark:
4   x: 1(navigation mode; 2(mapping mode; model:3(incremental mapping mode;
5     After starting up, the navigation system will actively report once;
6     The navigation system will automatically report when switching modes;

```

12. Toggle navigation mode

```

1 request: model: mode
2 response: model:x
3 mark:
4   | mode | x | remark |
5   | ----- | ----- | ----- |
6   | navi | 1 | navigation mode |
7   | mapping | 2 | Mapping mode |
8   | remap | 3 | incremental mapping mode |

```

13. save map

```
1 request: save_map
2 response: model: 1
3 mark:
4     Called after the drawing is completed
```

14. switch map

```
1 request: call_web[apply_map:map]
2 response: apply_map[map alias]
3     apply_map: failed
4 mark:
5     map : map md5
6     alias : alias (if no alias response: apply_map[map])
7     failed : switch map failed
8     After the map switching is completed, the navigation will also be repositioned, and
the repositioning is completed and reported to `initpose:0, xy radian`, which can be used as
a sign that the map is switched;
```

15. Get the coordinates of the marked point

```
1 request: nav:get_flag_point[name]
2 response:
3     get_flag_point[x,y,radian,type,name]
4     get_flag_point: -1
5 mark:
6     x,y : coordinates
7     radian : radian
8     type : type
9     name : point name
10    -1 : The calibration point cannot be found
```

16. set calibration point

```
1 request: nav:set_flag_point[x,y,radian,type,name]
2 response:
3     set_flag_point:result
4 mark:
5     x,y : coordinates;
6     radian : radian;
7     type : charging pile type fixed: charge
8     Delivery point type: delivery
9     Product type: production
10    Recycling point type: recycle
```

```
11    Waypoint Type: normal
12    Others: Types can be customized according to business needs
13    name : point name
14    result:
15        0 : success
16        -1 : the name length is greater than 50
17        -2 : file parsing failed
18        -3 : point repetition
```

17. Delete calibration point

```
1  request: nav:del_flag_point[name]
2  response:
3      del_flag_point:result
4  mark:
5      name : the point to delete
6      result:
7          -1 : file not found
8          -2 : file parsing failed
9          -3 : point not found
```

18. reset

```
1  request:
2  nav:reloc[x,y,radian] (relocate at (x,y,radian) coordinates;
3  nav:reloc_name[point] (relocate at the point of the specified name;
4  response: initpose:0, xy radian
5  mark:
6  The navigation system will actively report when starting up/restarting the
  navigation/switching the map (in these cases, the navigation will be relocated;
7  xy radian: machine coordinates;
```

19. Absolute relocation

(Note: Sending a name to relocate obtains the point position in the "calibration position" mode)

```
1  request:
2      nav:reloc_absolute[point name]
3  response: initpose:0,x y radian
4  mark:
5      point name:point name;
6      Do not match the map environment, and force relocation to the specified point;
```

20. Absolute position relocation (coordinate relocation)

(Note: Only supports RSNF-3.2.2 and above)

```
1 request:
2     nav:reloc_abpoint[x,y,radian]
3 response: initpose:0,x y radian
4 mark:
5     point name:point name;
6     Do not match the map environment, force relocation to the specified coordinates;
```

21. move robot

```
1 request:
2     move[distance,angle] [int, int] type
3     move[distance,angle,speed] [int, int, double] type
4 response: move:done:xx
5 mark:
6     1. One of distance and angle must be 0, that is, the robot can only walk in a straight
7       line or turn, and cannot walk in an arc
8     2. If move[0,0] is sent, it means stop moving
9     3. The unit of distance is mm
```

```

9      4. The unit of angle is Euler angle, and the value range is between [-180,180]
10     5. speed is the speed parameter, if not specified, the robot will walk at the default
    speed (0.3 meters per second for a straight line, 40 degrees per second for a turning angle)
11     Second)

12     6. speed must be greater than 0, otherwise it will be processed according to the
    default speed
13     7. If speed is specified and distance is not 0, speed represents the speed of walking in a
    straight line, and the unit is m/s
14     8. If speed is specified and angle is not 0, speed represents the speed of the corner,
    and the unit is degree/second
15     9. Reply to move:done:16: Go straight and complete
16     10. Reply to move:done:17: left turn completed
17     11. Reply to move:done:18: turn right is completed
18     12. move:done:19: Backward completed
19     13. move:done:20: Cancel
20     14. There is an obstacle: move_status:6

```

22. Navigate given target point name

```

1  request: nav_point[foreground]
2  response:
3      Refer to `ROS active reporting - 2. Navigation status`
4  mark:
5      When the sending target point type is `charge`, connect to the charging pile after
    successful navigation
    The Chinese target point needs to be set under the "Calibrate Position" option on the
6  web page
    Chinese character encoding method utf8, non-Chinese character encoding method
7  ASCII encoding

```

23. coordinate navigation

```

1  request: goal:nav[x,y,radian]
2  response:
3      Refer to `ROS active reporting - 2. Navigation status`
4  mark:
5      x,y are coordinates, radian is radians

```

24. pause navigation

```

1  request: nav_pause
2  response:
3      Refer to `ROS active reporting - 2. Navigation status`

```

25. restore navigation

```
1 request: nav_resume
2 response:
3 Refer to `ROS active reporting - 2. Navigation status`
```

26. cancel navigation

```
1 request: nav_cancel
2 response:
3 Refer to `ROS active reporting - 2. Navigation status`
```

27. Get the current coordinates

```
1 request: nav:get_pose
2 response:
3     nav:pose[x,y,radian] current robot coordinates
4     nav:pose:notfound The robot is positioning
5 mark:
6     x, y, radian are floating point numbers with two decimal places, where x, y are
    coordinates, radian is radian
```

28. Modify the navigation maximum speed

```
1 request: max_vel[0.6]
2 mark:
3     Range: [0.3,1.0]
4     Used to adjust the maximum speed of navigation
```

29. charging docking

```
1 request: dock:start
2 response:
3 Refer to `ROS active reporting - 3. Core status`
```

30. Undock

```
1 request: dock: stop
2 response: none
3 mark: This command is valid during the backward process of docking the charging pile
```

31. Get battery fixing information

```

1 request: get_battery_info
2 response:
3     battery_info{manufacturer nominal_voltage temperature cycle_times rated_capacity
full_capacity capacity health}
4 mark:
5     manufacturer : manufacturer
6     nominal_voltage : nominal voltage
7     temperature : temperature
8     cycle_times : cycle times
9     rated_capacity : rated capacity
10    full_capacity : full capacity
11    capacity : current capacity
12    health : health degree

```

32. Get battery dynamic information

```

1 request: get_current_info[open?1:0]
2 response: current_info{55 202 0 0 1}
3     //Voltage current adapter current warning offline
4 mark:
5     open: 1 : open; 0 : close;
6     voltage : voltage;
7     current : current;
8     adapter_current: adapter current; (not yet implemented, requires hardware
support)
9     warning : warning;
10    offline : 0: offline 1: online;

```

33. Get navigation dynamic parameters

```

1 request: update_dynamic
2 response:
3     get_max_vel: speed
4     get_stop_time : second
5     get_global_p : p
6     get_max_plan_dist :width
7     get_stop_nearby : state
8     get_use_speed_slow : m
9 mark:
10    speed : current navigation speed
11    second : the time to stay when encountering an obstacle
12    p : whether the global path considers temporary barriers 1: true consider; 0: false
13    width:fixed route obstacle avoidance width
14    state:nearby parking status
15    m:whether to adjust the minimum speed according to the curvature of the route

```

34. write navigation speed

1 request: write_max_vel[0.6]

2 response:

get max vel:0.6 : where 0.6 is the current maximum speed

3 Mark:

Range: [0.3,1.0]

Used to adjust the maximum speed of navigation

```
2 response:
3     get_max_vel:0.6 : where 0.6 is the current maximum speed
4 mark:
5     Range: [0.3,1.0]
6     Used to adjust the maximum speed of navigation
```

35. Set the dwell time when encountering obstacles

```
1 request: set_stop_time[5]
2 response:
3     get_stop_time:5.0
4 mark:
5     Range: [1,10]
```

36. Set whether the global path considers temporary obstacles

```
1 request: set_globalcost_p[1]
2 response:
3     get_global_p:1.0
4 mark:
5     1 : consider temporary barriers;
6     0 : do not consider temporary barriers;
```

37. Automatically report coordinate control

```
1 request:nav:get_pose[open?on:off]
2 response:
3     nav:pose[x,y,radian] current robot coordinates
4     nav:pose:notfound The robot is positioning
5 mark:
6     open: on: open; off: close
7     x, y, radian are floating point numbers with two decimal places, where x, y are
    coordinates, radian is radian
```

38. Set the position and orientation of other robots on the local map

```
1 request: robot_cost[x,y,z,line_speed,hostname,radius]
2 mark:
3     x,y : coordinates;
```

```
4    z : radians;
5    line_speed : line speed;
6    hostname: machine name;
7    radius : expansion radius (it is recommended to set 0.03 when the machine is
stationary, and 0.04 when the machine is moving);
```

39. Set CPU high performance mode

```
1  request: cpu_performance
```

40. Set the nearest stop

```
1  request: set_tolerance[1/0]
2  mark:
3      1: Enable nearby docking;
4      2: Close the nearest stop;
```

41. Get the special area where the robot is currently located

```
1  request: sendWeb[in_polygon]
2  response: in_polygon:special_area_name
3  mark:
4      When in a special area in_polygon:special_area_name;
5      When not in the special area in_polygon;
```

42. Take a temporary stop

```
1  request: get_nearest
2  response: answer_nearest{nofind/x,y,radian}
3  mark:
4      nofind: cannot find the stop point;
5      x,y are coordinates, radian is radians;
```

43. Get the ROS host startup time

```
1  request: get_poweron_time
2  response: power_on_t:yyyy-MM-dd HH:mm:ss
```

44. Filter Laser & 3D Noise Range

```
1 request: lidar_min[width]
2     mark:
3         width: width, floating point type, unit: meter;
```

45. Set whether the robot rotates (when the path, target point, etc. are blocked)

```
1 request: avoid_obstacle[on/off]
2     mark:
3         on: rotate;
4         off: no rotation;
```

46. Set the robot length and width

```
1 request: footprint[width,length]
2     mark:
3         width: width value, floating point type, unit: meter;
4         length: length value, floating point type, unit: meter;
5         Default value: footprint[0,0];
```

47. Set the robot length and width(3.2.6)

```
1 request: footprint[front,left,back,right]
2 mark:
3     front,left,back,right: the distance from the center of the machine to the front, left,
    back, right, floating point type, unit: meter;
```

48. Set the robot radius

```
1 request: robot_radius[radius]
2     mark:
3         radius: radius value, floating point type, unit: meter;
4         default value: robot_radius[0]
```

49. Set the direction and distance when the robot moves away from the QR code

```
1 request: agv_move[distance]
2     mark:
3         distance:
4             0: the machine leaves directly;
5             positive number: the machine starts navigating after moving forward a
specified distance, floating point type, unit: meter;
6             negative number: the machine starts navigating after moving backward
a specified distance, floating point type, unit: meter;
7             default value: 0.7;
```

50. Set the direction of the machine when navigating with a QR code

```
1 request: agv_mode[direction]
2     mark:
3         0: The machine performs QR code navigation in reverse;
4         1: The machine performs QR code navigation in forward direction;
5         Default value: 0;
```

51. Fixed route navigation

```
1 request: points_path[target_point_name]
2 response:
3     Refer to `ROS Active Report - 2. Navigation Status`
4     mark:
5         target_point_name:target point;
```

52. Cancel tag code navigation

```
1 request: agv:stop
2 mark:cancel tag code navigation
```

53. Query fixed route path

```
1 request: get_defined_plan[point]
2 response: getplan_dij:nofind
3     getplan_dij:path_point1 path_point2 path_point3...
4 mark:
5     point: navigation target point
6     nofind: unable to generate route
7     path_point: route point
```

54. Fixed waypoint navigation

- 1 request: list_point[1.12,2.13,3.11,4.13,5.11,2.01]
- 2 list_point_pre[1.12,2.13,3.11,4.13,5.11,2.01]
- 3 mark: Combine the path points into a fixed route and navigate along the route. The host computer sends data through the serial port. Since the maximum length of the data is 256,
- 4 the coordinates of a point (x, y, theta) occupy at least 15 bits
- 5 Each data retains two decimal places (retains precision)
- 6 Therefore, a maximum of 14 coordinate points are transmitted at a time. The first point is the current coordinate, and the last point is the target point coordinate.
- 7 Ensure that the path does not pass through obstacles or sharp turns at small angles. The distance between path points can be larger in straight line space and smaller in complex environments, the distance is a little smaller.
- 8 Before sending, please calculate the length of the string, please do not exceed 230
- 9 If the route is long, more than 14 coordinate points, it needs to be sent in packets. The last packet uses the list_point command, and the previous routes use the list_point_pre command

55. Actively issue speed control

- 1 request: app_vel[0.5,0.3]
- 2 mark: The first digit is the linear velocity, and the second digit is the angular velocity, both of which have positive and negative values. Positive numbers represent forward or left turn, and negative numbers represent backward or right turn.
- 3 Please ensure that the frequency of this interface is around 10hz
- 4 When starting debugging, please start from 0.1 and gradually increase to the required test value, otherwise the speed may be too high and affect safety.

56. Set the fixed route obstacle avoidance width

- 1 request: max_plan_dist[width]
- 2 mark: width: unit: meter

57. Generate a route to a specified point

```
1 request:get_plan_name[point]
2 get_plan_point[x,y,radian]
3 response:get_plan:error
4 get_plan:x1,y1,radian1,x2,y2,radian2...
5 get_plan1:x1,y1,radian1,x2,y2,radian2...
6 mark: get_plan_name[point] This interface can only transmit point names that are not in
  fixed route mode;
7     get_plan:error Failed to generate the route, usually because there are obstacles on
  the target point or path;
8     get_plan:x1,y1,x2,y2,... get_plan: is followed by the route coordinates
9     When the route is long, it will be reported in packets. The first packet starts with
  get_plan:, and the subsequent data packets start with get_plan1:.
10    If the route is not reported, it ends with +;
```

58. Set whether the depth camera participates in dynamic obstacle avoidance (RSNF1 3.2.3)

```
1 request: pc_switch[switch]
2 mark: switch is 0 to turn off the depth camera obstacle avoidance, and 1 to turn on the
  depth camera obstacle avoidance
```

59. Set whether to adjust the minimum speed according to the curvature of the route (RSNF1 3.2.4)

```
1 request: use_slow[switch]
2 mark: switch If 1, use the dynamic minimum speed; if 0, do not use the dynamic
  minimum speed
```

60. Set the move to target point (RSNF1 3.2.5)

- 1 request: move_goal[x,y,yaw]
- 2 response: move:done:15
- 3 mark: The parameter is the coordinate of the target point. Please keep two decimal places. Move forward at a speed of 0.1 and rotate at a speed of 0.1. Notify when reaching the target point
- 4 If there is an obstacle 10cm ahead, it will be considered to have arrived

61. Set whether local path obstacle avoidance takes map layer into account (RSNF1 3.2.5)

- 1 request: local_static[on]
- 2 mark: The parameter is on or off, on means using the map, off means not using the map
- 3 Considering the map layer will be safer, but correspondingly, due to the change in wall thickness, the width requirement is larger

62. Start docking QR code

- 1 request: agv:start[point_name]
- 2 mark: point_name: the point name of the QR code to be docked;

63. Query robot type

```
1 request: robot_type
2 response: robot_type:type
3 mark: type:
4     1: Hussar chassis
5     2: Moonknight chassis
6     3: How wheels chassis
7     4: Flybot with label code
8     5: Big dog
9     6: Flybot without label code
10    7: Flybot (double lidar)
11    8: Big dog (double lidar)
12    Snail sweeper and Forklift determine the robot type by ros version number
```

64. ROS wifi switch

```
1 request: robot_type
2 response: robot_type:type
3 mark: type:
4     1: Hussar chassis
5     2: Moonknight chassis
6     3: How wheels chassis
7     4: Flybot with label code
8     5: Big dog
9     6: Flybot without label code
10    7: Flybot (double lidar)
11    8: Big dog (double lidar)
12    Snail sweeper and Forklift determine the robot type by ros version number
```

65. Set anti-fall parameters

```
1 request: cliff_params[height,range,num,frequency]
2 mark: height: anti-fall height
3       range: anti-fall distance
4       num: the number of points that meet the anti-fall triggering requirement
5       frequency: filter anti-fall frequency
```

66. Set QR code obstacle avoidance distance

```
1 request: agv_obs_dist[fb]
2 mark: fb: Obstacle avoidance distance (unit: m, obstacle avoidance is triggered when the
        obstacle is less than this distance)
```

67. Reflector docking

```
1 Pickup Point Docking:
2 request: command:start[x,y,theta]
3       mark:
4           x: Vertical offset
5           This refers to the distance the machine travels into the shelf; 0 indicates that the
6           robot' scenter and reflector are at the same level.
7           y: Horizontal offset
8           theta: Angular offset
9 Stop Docking:
10 request: command:stop
11 Unloading Point Docking:
12 request: command:start_pose[x,y,theta]
13       mark:
14           x: Vertical offset
15           y: Horizontal offset
16           theta: Angular offset
17 response: laser_tag:data
18       mark:
19           data:
20               1:Docking in progress
21               0: Docking successful
22               241: Obstacle ahead
23               242: Reflector not detected
```

68. Set the reflector docking width

```
1 request: docking_width[width]
2 mark: width: distance between the two reflectors (unit: m)
```

69. After the robot lifts the shelf, the laser blocks the visible area

```
1 request: obs_angle[range_max, front, back, left, right]
2 mark: range_max`: Maximum distance for laser shielding (unit: m)
3 front: Front visibility angle (default: 55°)
4 back : Back visibility angle (default: 30°)
5 left: Left visibility angle (default: 30°)
6 right: Right visibility angle (default: 30°, radiating outwards from the robot's center)
```

Unique to Snail Sweeper

1. Set anti-fall parameters

```
1 request: clean_room[start:areaName]
2     mark:
3         areaName: clean area name
4 response:
5     clean_room[start:code,clean_id]
6     mark:
7         code:
8             -1: failed, room name not found;
9             -2: failed, unable to generate any route, starting point has obstacles or area
               is less than 1 square;
10            -3: failed, room not found, at least three points;
11            -4: failed, calculating route, unable to execute
```

Unique to Forklift

1. Take the pallet (adjust the lateral position deviation)

- 1 request: pallet:start[x,y,radians]
- 2 mark: x,y,radians are the coordinates of the pallet point

2. Place the pallet (adjust the lateral position deviation)

- 1 request: unload:start[x,y,radians]
- 2 mark: x,y,radians are the coordinates of the pallet point

3. Entering the elevator (adjusting the lateral position deviation before entering the elevator)

- 1 request: elevator_in:start[x,y,radians]
- 2 mark: x,y,radians are the coordinate of the elevator inside point

4. Exit the elevator

- 1 request: elevator_out:start[x,y,radians]
- 2 mark: x,y,radians are the coordinate of the elevator out point

5. Control Forklift arms

- 1 request: forklift_arm[1/0]
- 2 mark: 1: raise the forklift arm; 0: lower the forklift arm;

6. Exit the pallet docking state

- 1 request: pallet:stop

7. Set obstacle avoidance distance

- 1 request: avoid:distance[0.3,0.3,0.2,0.2]
- 2 mark: The parameters are front, back, left, and right;

8. Set obstacle avoidance distance

- 1 request: goal_back_point[x,y,theta]
- 2 response: same as normal navigation
- 3 mark: The forklift will not adjust the angle when reversing, so need to turn the rear end of the vehicle toward the goal point first;

9. Setting the laser angle at the forklift arm

```
1 request: set_arm_laser[angle]
2 mark: angle range:[20,160]
```

10. Detect whether there are obstacles at the target point

```
1 request: check: area_reachable[positionCenter[0],positionCenter[1],positionCenter[2],distanceToFront,distanceToBack,distanceToLeft,distanceToRight]
2 response: area_reachable:x
3 mark: positionCenter is the coordinate of the target point;
4       distanceToXX is the distance to determine whether there is an obstacle within a certain range of the target point;
5       area_reachable:x: x is 1, indicating no obstacle, and 0 indicates an obstacle;
```

11. Detect whether the robot can rotate safely

```
1 request: check: turn_angle[angle]
2 response: turn_angle_check:x
3 mark: angle is the angle that the robot will rotate (left rotation is positive, right rotation is negative), for example -90 means right rotation 90°;
4 turn_angle_check:x: When x is 1, it means that the vehicle can rotate, and 0 means that there is an obstacle and the robot cannot rotate;
```

12. Obtain the coordinates of the card plate detected by the camera and convert them onto the map

- 1 Request: get_pallet_pose[on/off]
2. Response: pallet_pose:[x,y,radian]
3. Mark: get_pallet_pose[on] enables the pallet coordinate reporting function, get_pallet_pose[off]` disables it;
4. When the reporting function is enabled, the navigation system will periodically report x, y, and radian if it obtains camera data; otherwise, it will not report any information;
5. x, y, and radian are all floating-point numbers with two decimal places, where x and y are coordinates, and radian is radians.

13. Set the pallet size

1. request: set_pallet_size[lenth_x,width_y]
2. mark:
 - lenth_x: Pallet fork insertion direction length: range ≥ 0 , floating-point type, unit m;
3. width_y: Pallet width: range ≥ 0 , floating-point type, unit m.

14. Distance to nearest obstacle in each direction of the Forklift

1. request: obs_data[front,back,left,right,front_left,front_right]
2. mark: Distance to the nearest obstacle in each direction (unit: cm)
3. obs_data[front,back,left,right,left-front,right-front]

15. Set up obstacle avoidance in 3D

- 1 request: obs_3d[switch]
- 2 mark:
- 3 switch:
- 4 on: Trun on 3D obstacle avoidance;
- 5 off: Turn off 3D obstacle avoidance;

Forklift pallet placement process

- 1 After reaching the designated position, take the pallet
android send to ros -> pallet:start[x,y,radians]
- 2 Start adjusting the lateral deviation
forklift{1 0 1 0 0} <- ros send to android
- 3 The lateral deviation adjustment is completed
forklift{1 0 3 0 0} <- ros send to android
- 4 Android sends to lower the forklift arm
android send to ros -> forklift_arm[0]
- 5 After lowering the forklift arm, start docking the pallet
forklift{0 0 3 0 0} <- ros send to android
- 6 The docking of the pallet is completed
forklift{2 0 2 0 0} <- ros send to android
- 7 Raise the forklift arm
android send to ros -> forklift_arm[1]
- 8 The forklift arm has been raised
forklift{1 0 0 0 0} <- ros send to android
- 9 Leave the pallet position
android send to ros -> move[1400,0]
- 10 Move completed
move:done:16 <- ros send to android
- 11 mark:move[1400,0] means moving forward 1.4m. This is for reference only. The actual distance between the pallet point and the pallet docking point should be calculated.

Transparent command

- 1 request: send_to_base[data]
- 2 response: base_data:{}
- 3 mark: navigation forwarding, sent to the power board

36V Power Control

- 1 request:
- 2 Open: The public power board needs to send the command first:
 send_to_base[208 12 02 01 98 97 116 116 101 114 121 95 49 00]
- 3 send_to_base[208 16 02 01 103 101 110 101 114 97 108 95 112 111 119 101
 114 00]
- 4 Close: send_to_base[208 16 02 00 103 101 110 101 114 97 108 95 112 111 119
 101 114 00]

24V Power Control

- 1 request:
- 2 Open: send_to_base[208 06 02 01 50 52 118 00]
- 3 Close: send_to_base[208 06 02 00 50 52 118 00]

Power Board Upgrade

1. Upload upgrade file

- 1 The web side will upload the upgrade file;

2. enter loading mode

- 1 request: base_upgrade[start]
- 2 response: base_upgrade:loading

3. upgrade

- 1 request: base_upgrade[affirm]
- 2 response: base_upgrade: success

test odom

- 1 roslaunch yoyo_node test_get_odom.launch
- 2 rostopic echo /mobile_base/debug/odom_pose2d

test imu

- 7 roslaunch yoyo_node test_get_yaw.launch
- 8 rostopic echo /mobile_base/debug/imu_angle